

Review paper

Phases of compiler design

The six-phase model, measured against Zero — a compiler that reports its own phase structure as machine-readable data.

Submitted by

Harshit Khemani

Co-authors

Kush Ahuja, Mohit Kumar Mishra, Kushagra Agrawal

Submitted to

Ms. Ankita Sharma



Read online

`zero.khe.money`

Source and dataset at

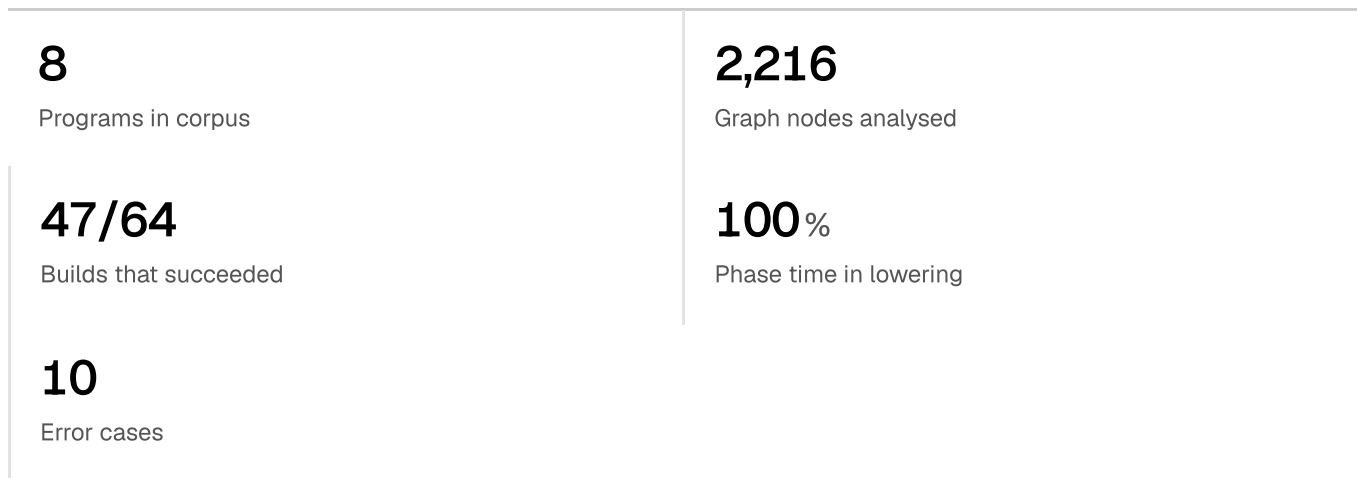
github.com/HKTITAN/phases-of-zero.

Abstract

Compiler construction is conventionally taught as a pipeline of six phases: lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimization, and target code generation, with symbol-table management and error handling running alongside all six. This decomposition is a teaching abstraction. Production compilers rarely expose it, so students seldom see the phases as separable, measurable objects.

This paper reviews that model against Zero (<https://zerolang.ai/>) 0.3.4, an experimental graph-first systems language from Vercel Labs whose compiler reports its own phase names, per-phase timings, symbol tables, and diagnostics as structured JSON. We construct a corpus of 8 Zero programs (680 lines, 3,411 tokens, 2,216 graph nodes) and a second corpus of 10 deliberately malformed programs, then measure the compiler across 64 program-target build combinations.

We report four findings. First, the phases do not disappear in a graph-first design — they *relocate*. Lexical, syntactic and semantic analysis move to an ingestion boundary that admits programs into a persistent graph, after which the compile path performs only lowering, code generation and linking. All 7 front-end error cases were refused at that boundary, and in none of them did the malformed program enter the graph store. Second, lowering accounts for effectively all measurable phase time (100%); the entire front end runs below the compiler's 1 ms reporting resolution because it reads stored facts instead of recomputing them. Third, the front end and the back end accept *different languages*: 17 of 64 builds failed with BLD004 on programs that had already passed `zero check`, and the compiler reports `ok: true` alongside `buildable: false` in the same document. Fourth, and against the language's own premise, its structured output is far more expensive to read than its prose — `zero check` prints four bytes where `zero check --json` returns tens of thousands.



All figures produced by `tools/capture.mjs` against Zero 0.3.4 (build 5b3a90a) on 13th Gen Intel(R) Core(TM) i5-13450HX, 16 cores, win32/x64. Captured 2026-08-09.

1. Introduction

Every undergraduate compilers course opens with the same diagram: source text enters at the top, passes through a vertical stack of labelled boxes, and machine code emerges at the bottom. Two narrow rectangles run down the side of the stack, touching every box — the symbol table and the error handler. The diagram is due in its modern form to Aho, Lam, Sethi and Ullman (<https://www.pearson.com/en-us/subject-catalog/p/compilers-principles-techniques-and-tools/P200000003472>), and it has organised the field for four decades.

The diagram is also, in an important sense, unfalsifiable by students. Real compilers fuse phases for speed, interleave them for incrementality, and expose none of the boundaries. A student who runs `gcc` or `rustc` sees a binary appear and, on failure, a paragraph of English. The phases are real, but they are not *visible*.

This paper asks a narrow question: **does the classical six-phase model still describe a compiler built on a fundamentally different substrate, and if not, how does it differ?** We answer it empirically — by building a corpus, instrumenting the compiler through its own reporting interfaces, and measuring.

2. What Zero is

Zero (<https://zerolang.ai/>) is an experimental systems programming language released by Vercel Labs in May 2026 under Apache-2.0, with source on GitHub (<https://github.com/vercel-labs/zerolang>). It compiles to standalone native executables, has no garbage collector, gives explicit control over memory, and sits in roughly the design space of C and Zig. Its compiler core is written in C. Programs use the `.0` extension. We study version 0.3.4, build 5b3a90a.

Its distinguishing premise is stated in its own tagline: *the programming language for agents*. Zero is built on the assumption that AI agents, rather than humans, will be the primary consumers of compiler output — and the design follows that assumption further than any other language we are aware of.

```
p01_hello / src/main.0                                the smallest complete program

pub fn main(world: World) -> Void raises {
    check world.out.write("hello from zero\n")
}
```

Figure 1. `pub fn` exports. `World` carries runtime capabilities. `raises` marks the function fallible, and `check` propagates failure. This compiles to a 1536-byte native executable in 6 ms of lowering.

2.1 How it differs from current languages

Four departures matter for this paper.

The semantic graph is the program, not the text. A Zero package compiles from a binary `zero.graph` store holding declarations, types, calls, scopes, imports, capabilities and source-map facts as rows in sixteen relations. The `.0` files a human reads are a *projection* of that graph. In every other mainstream language text is authoritative and the compiler's internal representation is derived and discarded; Zero inverts that relationship (<https://zerolang.ai/concepts/semantic-vs-text>).

Diagnostics are data, not prose. Every command accepts `--json` against a versioned schema. Each diagnostic carries a stable code, a span, structured `expected` / `actual` facts, a `fixSafety` rating, and often a typed `repair` identifier.

Effects and capabilities are explicit and target-checked. A function performing I/O receives a `World` handle and is marked `raises`. Each target independently declares which capabilities it provides, so a program using the network is rejected at compile time for a target that declares no `net` capability.

The toolchain reports itself. A single `zero check --json` returns the phase list with timings, graph table row counts, the resolved call graph with contracts, cache keys with hit status, and a target readiness report. There is also `zero tokens`, which counts a program in LLM tokens.

2.2 Why its designers argue it is needed

The case Zero makes is about the cost of a translation layer. In a conventional agent loop, an agent writes text, the compiler renders its objection as English, and the agent parses that English back into an intent it already had. Zero's graph architecture documentation (<https://zerolang.ai/concepts/graph-architecture>) frames the contrast as two loops.

Traditional agent source loop

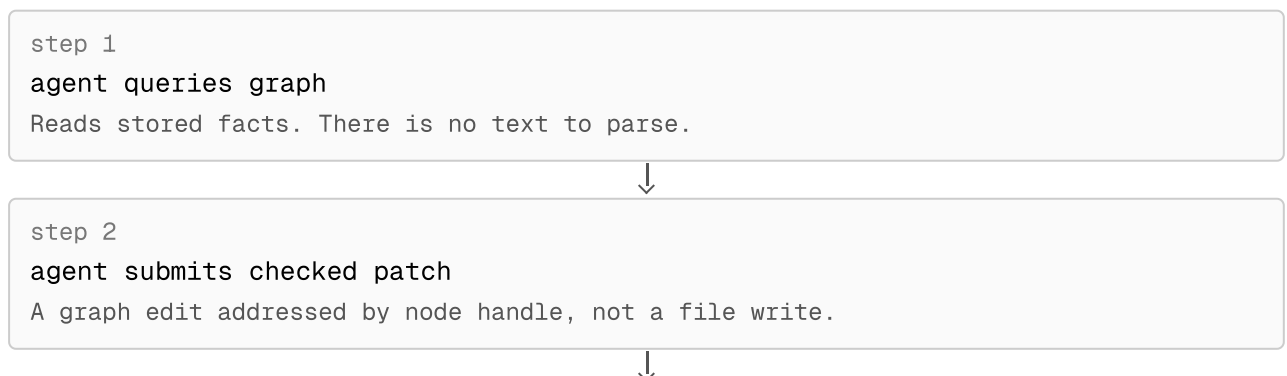
a cycle · 5 steps · repeats until the build passes

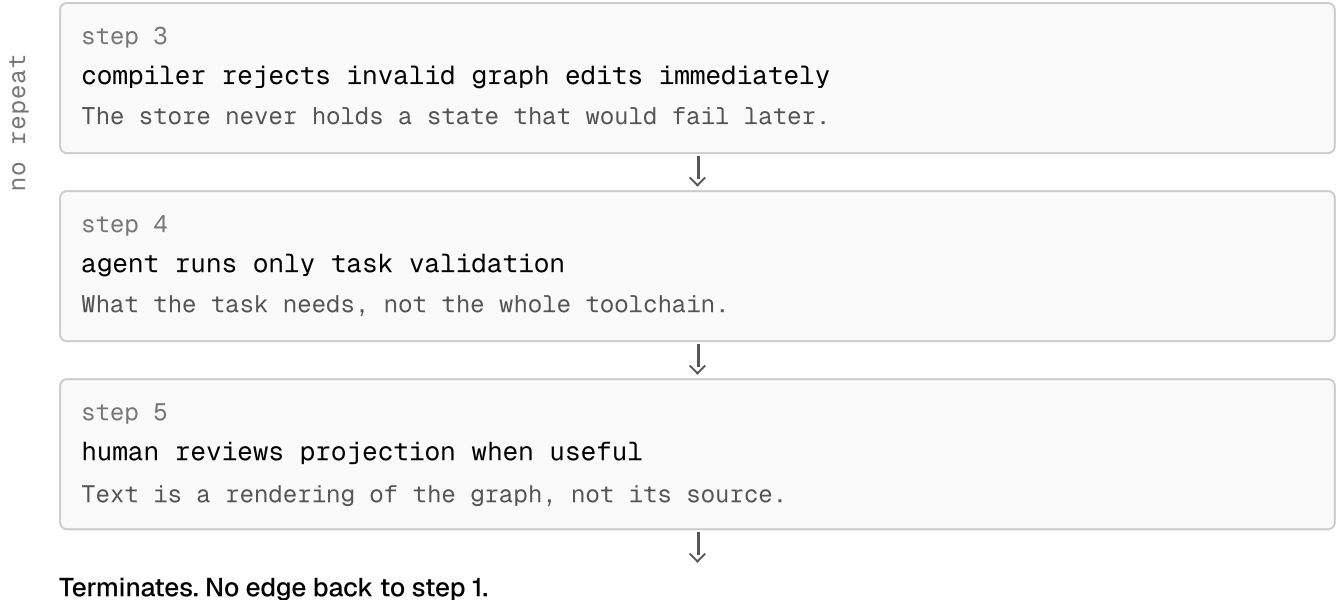


After step 5 the loop returns to step 1: (repeat). Every pass re-derives the same tokens, the same tree and the same bindings from text that has already been read, and the agent only learns an edit was invalid at the end of the pass.

Zero graph loop

largely linear · 5 steps · terminates





There is no (repeat) edge. An invalid edit is refused at step 3, at submission time, rather than discovered at the end of a build; the store therefore never holds a state the next pass would have to find and undo.

Three specific failures are claimed. *Line ranges are the wrong handle* — they drift the moment anything reformats, whereas semantic node identifiers do not. *English is a lossy encoding of compiler state* — the compiler knew the exact repair and discarded that structure to print a sentence. *Feedback latency bounds the loop*.

A fourth motive is arguably strongest and less often stated: **capability containment**. For code an agent wrote and no human read line by line, a compiler that refuses to build a program using a capability the target does not declare is a real safety property.

We record the counterarguments in the same breath. Structured diagnostics are not new: rustc (<https://doc.rust-lang.org/rustc/json.html>) and TypeScript have emitted machine-readable errors for years. And the adoption argument cuts hard against it — models write best in the languages that dominate their training data, so a language with a few thousand GitHub stars is one agents are, today, measurably worse at than Go or Rust. Zero is betting a tight verifiable loop outruns familiarity. That bet is unproven, and this paper does not settle it.

3. How traditional compilers work

The canonical decomposition treats compilation as a sequence of meaning-preserving translations. Each phase consumes one representation and produces the next.

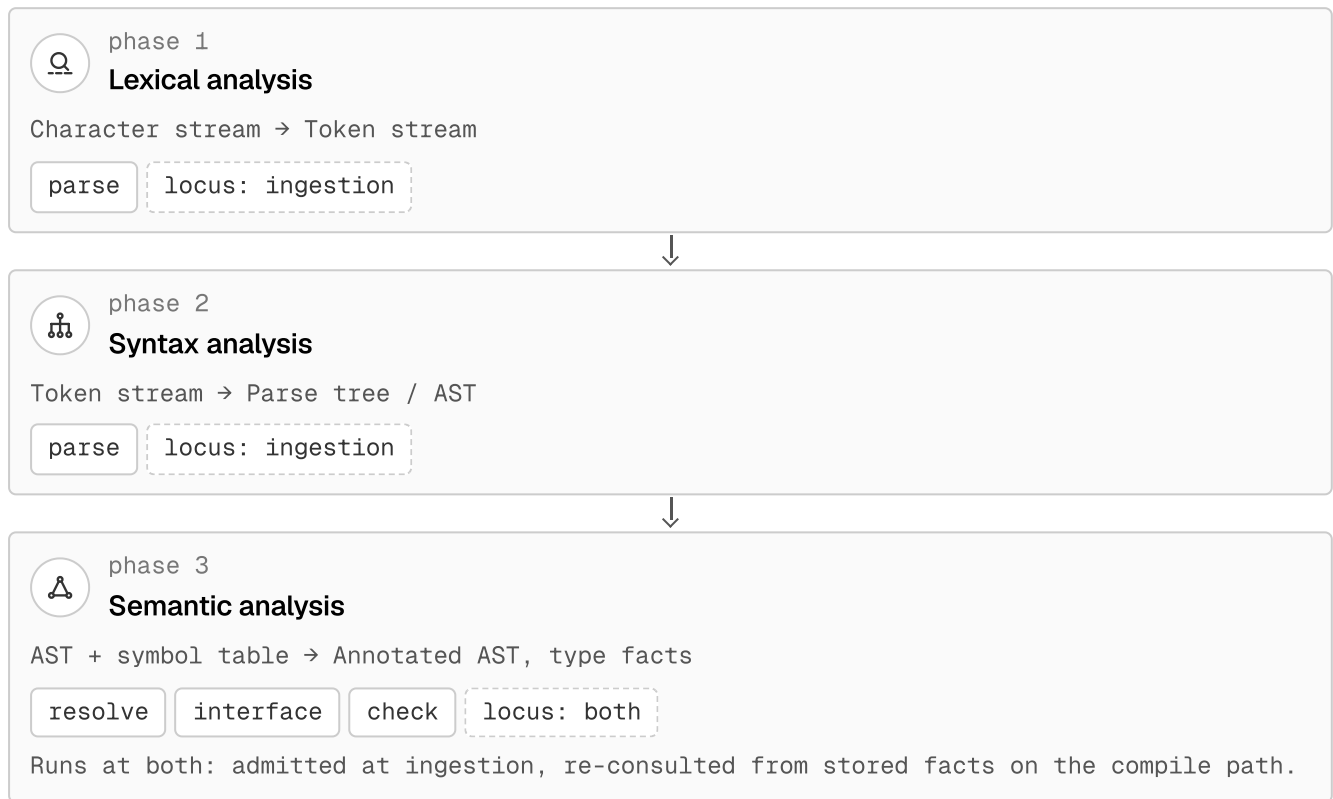
The **analysis** half — phases one to three, the front end — determines what the program means. Lexical analysis groups characters into tokens. Syntax analysis arranges tokens into a tree reflecting the grammar. Semantic analysis annotates that tree with types, resolves every name to a declaration, and rejects programs that are well-formed but meaningless.

The **synthesis** half — phases four to six, the back end — determines how the program runs: an intermediate representation, optimization over it, then instruction selection and object emission.

Two activities refuse to sit in any single box. The **symbol table** is written by the front end and read by every later phase. **Error detection** occurs in all six, and the phase that detects an error largely determines how good the message can be. That last point is the one we test in §6.4.

Phases 1–3 · at the ingestion gate

zero import · once per edit



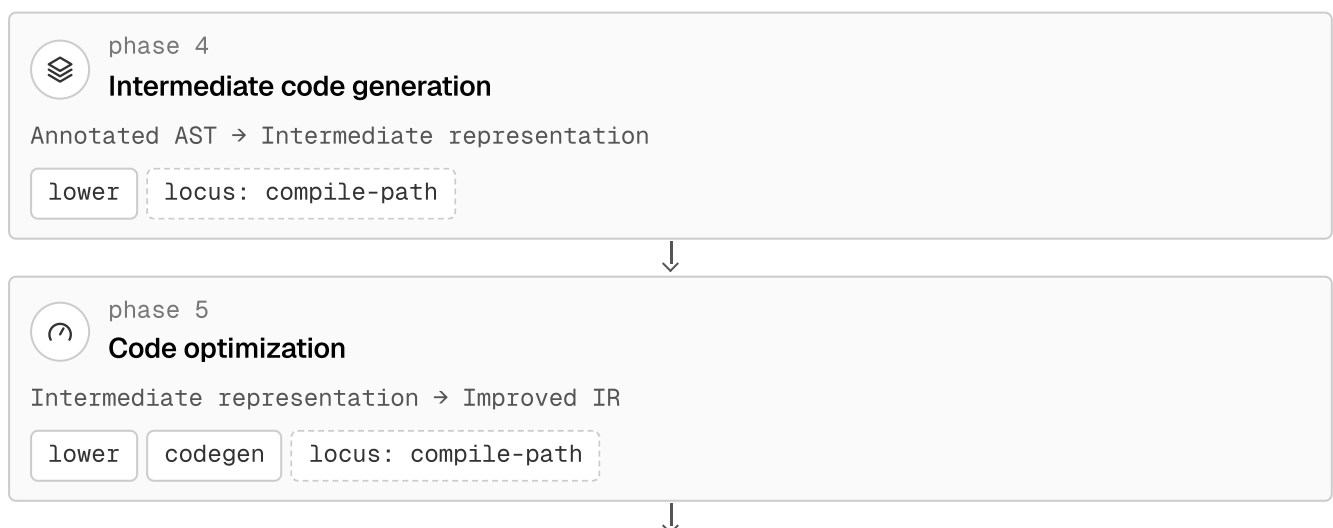
These three run when text enters the graph and are not re-run to produce a build. Phase 3 is the boundary case: it is admitted here and then re-consulted on the compile path from stored symbol, type and scope tables rather than rebuilt.

The admission boundary

above: once per edit · below: once per build

Phases 4–6 · on the compile path

zero build · once per build





phase 6

Target code generation

Optimized IR → Machine code / object

codegen

object

link

locus: compile-path

These three are what a build actually costs. They read a graph whose names are already bound and whose types are already recorded, which is why the compile path can begin at lowering.

cross-cutting · spans phases 1-6

Symbol table management

Classical

A data structure rebuilt by the front end on every compilation.

As Zero realises it

A persisted set of graph tables (schema, package, module, declaration, scope, import, symbol, type, effect, capability, ownership, resource, node, edge, projection, sourceMap) carried between runs in `zero.graph` and addressable by stable node handles.

```
probe: zero query --json --full
```

cross-cutting · spans phases 1-6

Error detection and reporting

Classical

Diagnostics rendered as prose for a human reader.

As Zero realises it

Structured records with a stable code, a span, expected/actual facts, a `fixSafety` rating and an optional typed repair id — designed to be consumed by a program, not parsed from English.

```
probe: zero check --json | zero explain --json | zero fix --plan --json
```

The classical six-phase stack, drawn once. The split is not a redrawing of the phases but a statement about cadence: phases 1–3 are paid when an edit is admitted, phases 4–6 when an artifact is requested. The two panels are the cross-cutting concerns the textbook draws as vertical bars beside the stack; in Zero both are persisted and directly inspectable, which is why the split above is observable rather than asserted. Phase numbering, input/output pairs, Zero phase names and locus values are read from `lib/phases.ts`. On a narrow screen the two bars move underneath the stack rather than beside it.

4. How Zero's compiler differs

Zero's own compile-path documentation (<https://zerolang.ai/concepts/compile-path>) states the contrast directly. A conventional compiler runs source files through a lexer and parser to an AST, then name resolution, type checking, IR lowering, optimization and codegen. Zero starts from the graph store.

Traditional parse-first path

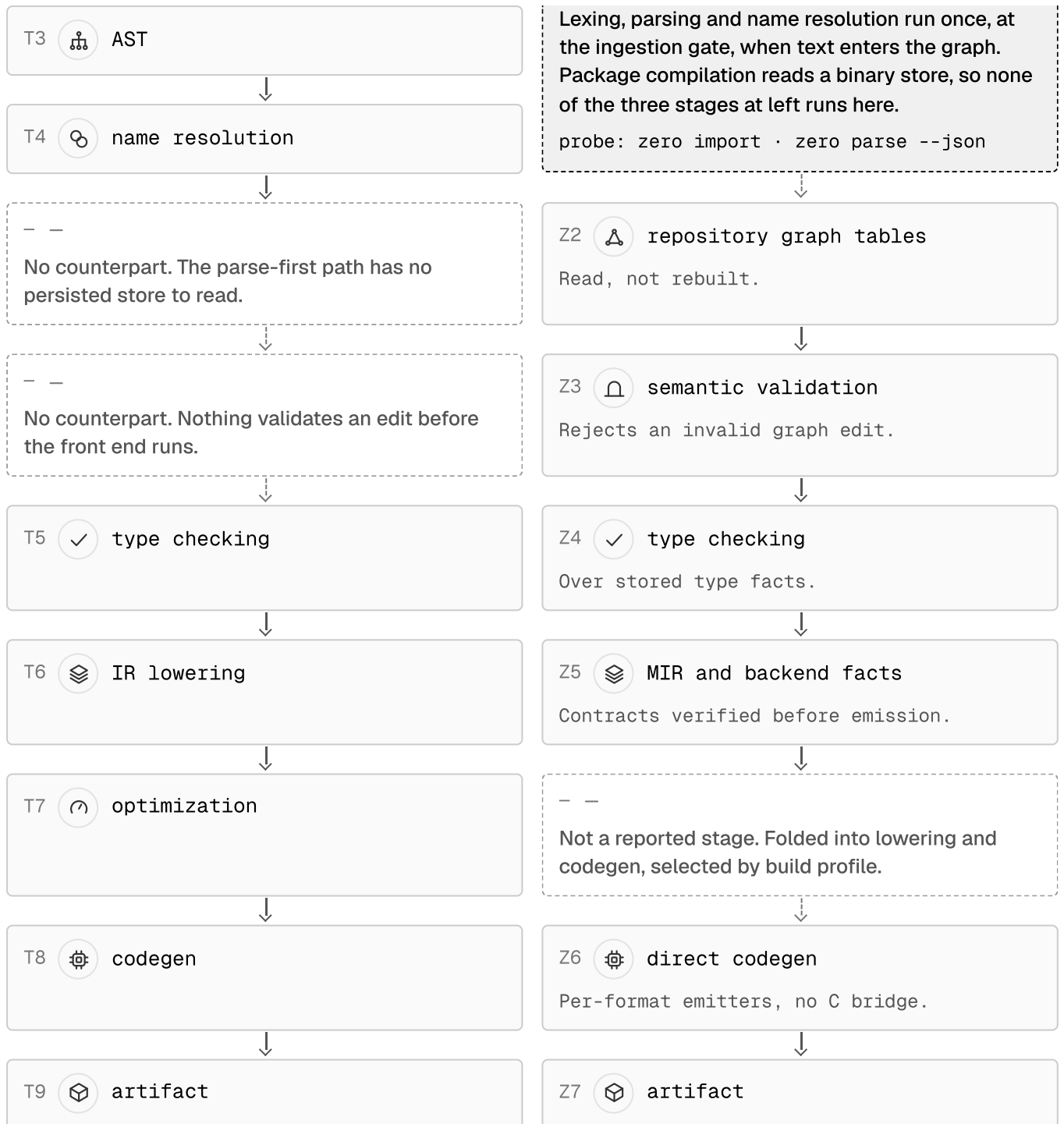
as documented for Rust, Go, Zig, C · 9 stages

T1  source filesT2  lexer/parser**Zero graph-first path**

Zero 0.3.4 package compilation · 7 stages

Z1  zero.graph

moved off this path



Both tracks read top to bottom. Slots are aligned, so a dashed box marks a stage one path has and the other does not: three traditional stages leave the Zero build path entirely, one more (optimization) is folded into two others, and two Zero stages have no traditional counterpart. Stage labels are the documented names, unabbreviated.

Text alternative to the diagram above: the two compile paths as ordered stage sequences, 9 stages against 7. Rows are positional only — row *n* holds stage *n* of each sequence and does not assert a correspondence between them. — means the sequence has ended. No units; this table is definitional.

#	Traditional parse-first stage	Zero graph-first stage
1	source files	zero.graph
2	lexer/parser	repository graph tables
3	AST	semantic validation
4	name resolution	type checking
5	type checking	MIR and backend facts
6	IR lowering	direct codegen
7	optimization	artifact
8	codegen	—
9	artifact	—

The consequential difference is what is *missing* from the second sequence. There is no lexer, no parser and no separate name-resolution stage on the compile path, because by the time a program is in the graph store its names are already bound and its types already recorded. Those stages still exist — they run at the boundary where text is admitted into the graph.

This is why the compiler reports `resolve before parse`, an ordering that reads as a typo until you understand the substrate. The eight phases it reports, in its own order:

- `resolve` — Binds names against stored symbol facts.
- `parse` — Graph-native; no character scan on the package path.
- `interface` — Public symbol surface and import graph fingerprinting.
- `check` — Types, effects, ownership, capability requirements.
- `lower` — Graph HIR to MIR, with contract verification before emission.
- `codegen` — MIR to target machine code via direct emitters.
- `object` — Object-file construction in the target format.
- `link` — The only phase the compiler marks non-cacheable.

And the two cross-cutting concerns become first-class inspectable objects:

Symbol table management. A persisted set of graph tables (schema, package, module, declaration, scope, import, symbol, type, effect, capability, ownership, resource, node, edge, projection, sourceMap) carried between runs in `zero.graph` and addressable by stable node handles. Probe: `zero query --json --full`

Error detection and reporting. Structured records with a stable code, a span, expected/actual facts, a fixSafety rating and an optional typed repair id — designed to be consumed by a program, not parsed from English. Probe: `zero check --json | zero explain --json | zero fix --plan --json`

5. Methodology

5.1 Corpus construction

We wrote 8 Zero packages of increasing size and feature coverage, from a six-line hello-world to a 304-line arithmetic tokenizer. Each was developed until `zero check`, `zero test` and `zero run` all succeeded, or until the obstruction was documented as a finding. The corpus totals 680 non-empty lines, 3,411 tokens and 23 passing test blocks. All eight are reproduced in full in §7.

5.2 Error corpus

Measuring where errors are detected requires programs that fail on purpose. We wrote 10 minimal packages, each violating exactly one rule and targeting a specific phase. Each carries a `case.json` declaring the phase it targets *before* measurement, so the mapping in §6.4 is a prediction rather than a post-hoc fit.

5.3 Instrumentation

One harness, `tools/capture.mjs`, drives every measurement and writes a single JSON document. Per program it captures the token stream, parse summary, full semantic report, graph, source mappings, phase timings and artifact measurements, then builds every program for every advertised target. We report phase timings from `zero time` rather than `zero check`, because the former drives the pipeline through emission; and wall-clock separately, as median of five runs, because ~40 ms of process startup dominates at this scale.

5.4 Reproducibility

Every number derives from one machine-generated dataset. No figure is transcribed by hand; the prose reads the same JSON the charts do. Appendix A gives the commands. The environment was an 13th Gen Intel(R) Core(TM) i5-13450HX with 16 logical cores and 31.7 GB of memory, running win32/x64.

6. Results

6.1 Mapping the classical phases onto Zero

Zero reports eight phases; the classical model names six. They do not correspond one-to-one, and the mismatches are informative.

The six canonical front-to-back compiler phases (Aho, Lam, Sethi and Ullman) against the phase names Zero 0.3.4 reports in its own `--json` output. One mapping row per phase, followed by a row stating where the realisation departs from the textbook. Counts and units: none — this table is definitional.

#	Classical phase	Input → output	Zero phases	Probe	Locus
---	-----------------	----------------	-------------	-------	-------

#	Classical phase	Input → output	Zero phases	Probe	Locus
1	Lexical analysis	Character stream → Token stream	parse	zero tokens --json	ingestion
Divergence — Runs only when text enters the system. Package compilation reads a binary graph store and never re-scans characters, so the scanner is absent from the steady-state path.					
2	Syntax analysis	Token stream → Parse tree / AST	parse	zero parse --json	ingestion
Divergence — The tree is not the compiler's working representation. Parsing exists to admit text into the graph; once admitted, structure is stored, not re-derived.					
3	Semantic analysis	AST + symbol table → Annotated AST, type facts	resolve interface check	zero check --json	both
Divergence — Split across three reported phases and persisted as typed graph facts. The symbol table is not rebuilt per compile — it is the stored `symbol`, `type` and `scope` tables.					
4	Intermediate code generation	Annotated AST → Intermediate representation	lower	zero size --json (loweredIrBytes)	compile-path
Divergence — Lowering runs graph HIR to MIR directly, and MIR contracts are verified before emission. The IR is never printed: <code>--emit llvm-ir</code> fails with BLD004 on every target, because the direct backend has no LLVM path. The only observable is its size in bytes.					
5	Code optimization	Intermediate representation → Improved IR	lower codegen	zero build --profile release-small tiny	compile-path
Divergence — Not a separately reported phase. Optimization is selected by build profile rather than exposed as a pass pipeline, so its cost is folded into lower and codegen.					
6	Target code generation	Optimized IR → Machine code / object	codegen object link	zero build --emit exe && zero size --json	compile-path
Divergence — Three reported phases, not one. Direct per-format emitters replace a C bridge, and only `link` is marked non-cacheable.					

Semantic analysis fragments into three reported phases because interface fingerprinting is separated out to drive incremental invalidation. Optimization has *no* reported phase: it is selected by build profile. And the IR is never printed — `--emit llvm-ir` fails with BLD004 on every target, leaving the lowered module's *size* as the only observable.

6.2 Where compile time actually goes

Across all 8 programs, 100% of reported phase milliseconds are spent in `lower`. Every other phase reports 0 ms.

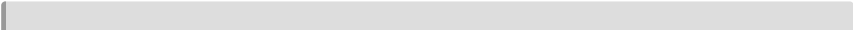
Where the milliseconds go

Pick a program, smallest to largest. The left panel is every phase time the compiler reported for it; the right panel is the size of the IR that lowering produced, placed against the whole corpus.

Program, by non-empty source lines

p01_hello 6 lines	p02_arith 31 lines	p07_generics 45 lines	p03_control 54 lines	p05_errors 54 lines	p04_shapes 85 lines
p06_memory 101 lines	p08_lexer 304 lines				

6 Non-empty source lines	6ms Total reported phase time
6ms Of which lower	2,185B Lowered IR bytes

Reported phase time		p01_hello · phases.coldTimed
resolve		0 ms
parse		0 ms
interface		0 ms
check		0 ms
lower		6 ms
codegen		0 ms
object		0 ms
link		0 ms

7 of the 8 phases returned 0 ms; the bar for each is an empty track with a tick at the origin, not a missing row. Only lower registered any time at all.

Lowered IR size across the corpus

8 programs · backend.size.loweredIrBytes

Lowered IR bytes (vertical) against non-empty source lines (horizontal), all 8 programs



Highlighted, drawn larger and ringed: p01_hello at 6 non-empty lines and 2,185 bytes of lowered IR.

Source size on the horizontal axis, the size of the MIR lowering produced on the vertical. The IR itself is never printed — `--emit llvm-ir` fails with BLD004 on every target — so its byte count is the only observable the compiler offers.

Every phase p01_hello reported, in the order the compiler lists them, from phases.coldTimed. Times are whole milliseconds as reported: a 0 means the phase returned inside one millisecond, not that it was skipped. Share is that phase as a percentage of the 6 ms total.

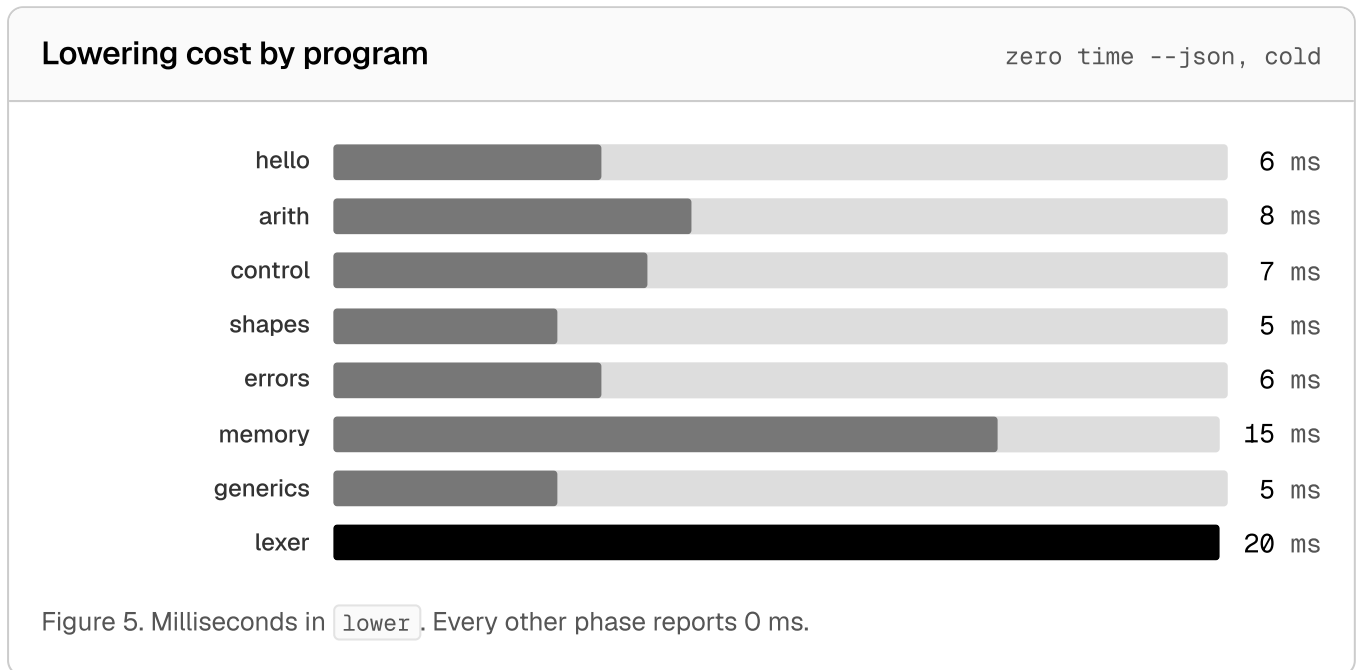
#	Phase	Reported	Share	Cacheable
1	resolve	0 ms	0%	yes
2	parse	0 ms	0%	yes
3	interface	0 ms	0%	yes
4	check	0 ms	0%	yes
5	lower	6 ms	100%	yes
6	codegen	0 ms	0%	yes
7	object	0 ms	0%	yes
8	link	0 ms	0%	no
All phases		6 ms	100%	—

The corpus ordered by non-empty source lines, smallest first, with the lowered IR size each program produced and the time its lower phase reported. Bytes per line is derived by division and is stated only to show the ratio is roughly stable. The selected program is marked in the first column.

Program	Non-empty lines	Lowered IR bytes	Bytes per line	lower
p01_hello · selected	6	2,185	364	6 ms
p02_arith	31	5,994	193	8 ms
p07_generics	45	6,573	146	5 ms
p03_control	54	10,340	191	7 ms
p05_errors	54	8,731	162	6 ms

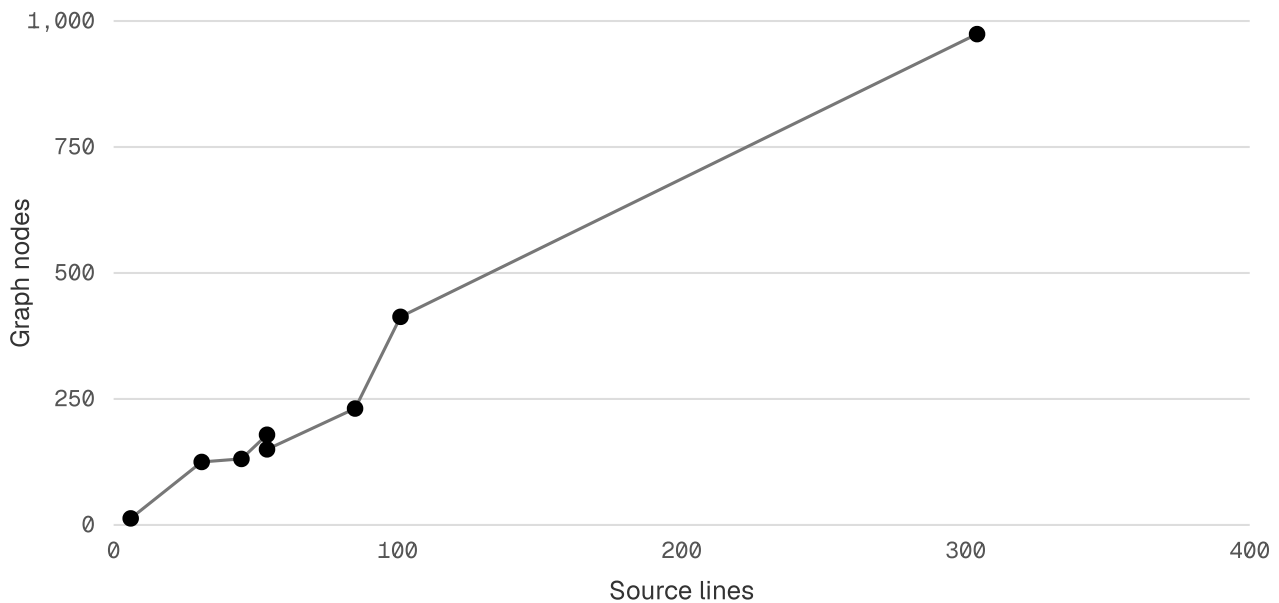
Program	Non-empty lines	Lowered IR bytes	Bytes per line	lower
p04_shapes	85	9,499	112	5 ms
p06_memory	101	25,416	252	15 ms
p08_lexer	304	52,110	171	20 ms

Read the two panels together: the front end costs nothing measurable because it reads facts already stored in `zero.graph` rather than re-deriving them from text, and `lower` is the only phase whose time, and whose output, grows with the size of the program.



Graph size against source size

8 programs

Figure 6. ≈ 3.3 graph nodes per source line — a constant-factor expansion, not a blow-up.

This is the expected consequence of the architecture. In a text-first compiler the front end reconstructs meaning from characters on every invocation. In Zero it reads a store where names are already bound, so it does almost no work. What remains expensive is the one phase that cannot be cached away.

The honest caveat is resolution: the compiler reports integer milliseconds, so "0 ms" means "under one millisecond", not "free".

Phase composition, largest program

p08_lexer · 304 lines

Figure 7. All eight reported phases. The bar is effectively one segment: `lower` at 20 ms.

6.3 The symbol table, persisted

In the classical model the symbol table is a structure the front end builds and discards. In Zero it is sixteen persisted relations, carried between invocations and reported as row counts on every compile.

All 8 corpus programs in corpus order, measured at each stage of the pipeline. Lines and bytes are source text; tokens are the total emitted by zero tokens including newlines; nodes, edges, declarations, symbols and types are row counts in the persisted graph; typed nodes and calls

come from zero check; artifact is the host-target executable in bytes, or — where the backend refused to build it.

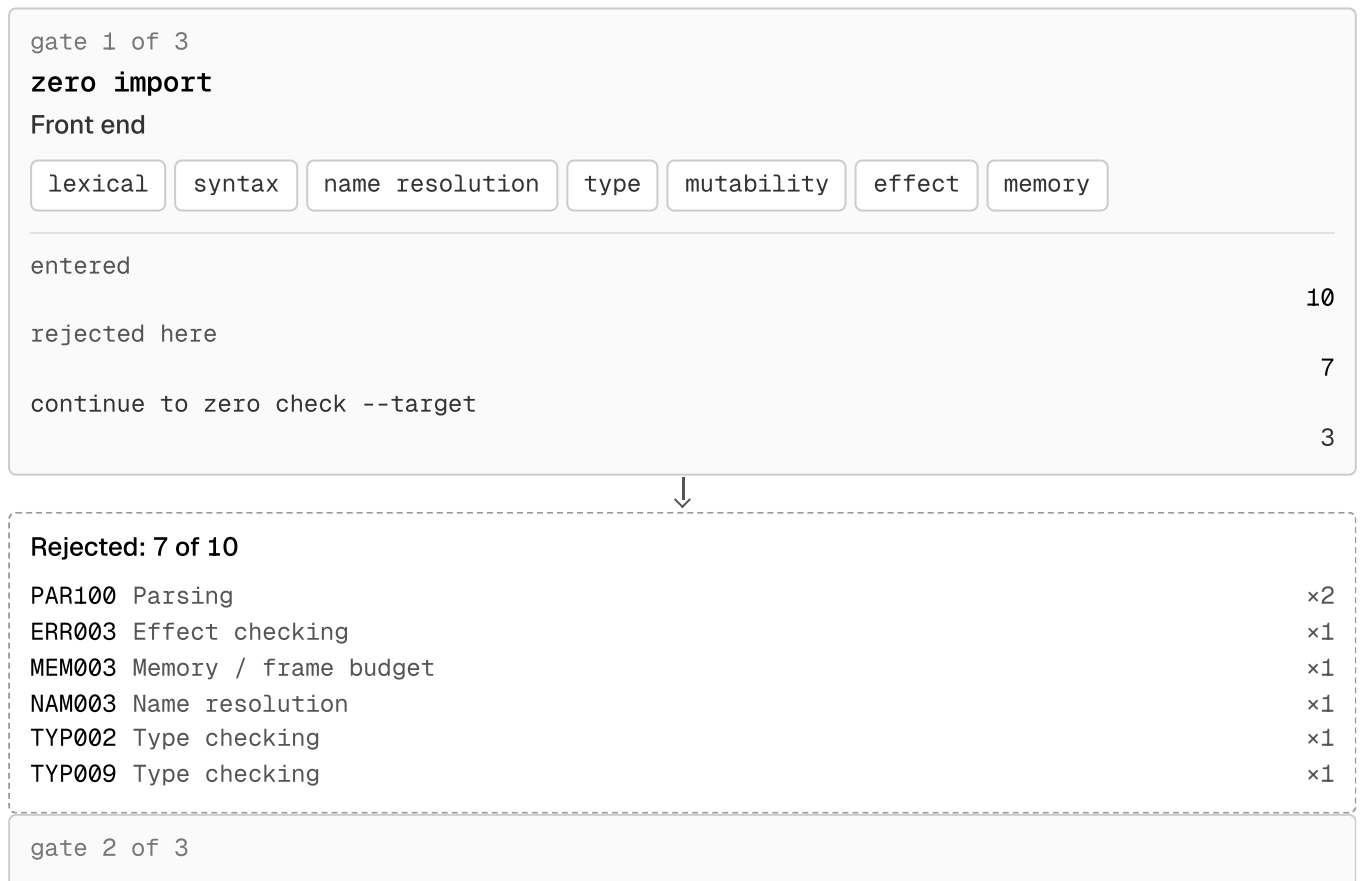
Program	Lines	Bytes	Tokens	Nodes	Edges	Decls	Symbols	Types	Typed nodes	Calls	Artifact bytes
p01_hello	8	140	41	13	12	2	3	5	5	1	1,536
p02_arith	40	781	219	125	123	15	16	29	29	19	2,048
p03_control	62	1,624	297	179	177	16	17	31	31	21	2,048
p04_shapes	102	2,478	471	231	229	33	34	55	55	24	2,048
p05_errors	66	1,984	292	150	148	20	21	35	35	19	—
p06_memory	123	3,518	786	413	411	49	47	96	96	45	3,584
p07_generics	56	1,298	277	131	129	20	21	37	37	20	1,536
p08_lexer	335	10,390	1,804	974	972	108	109	176	176	106	5,632

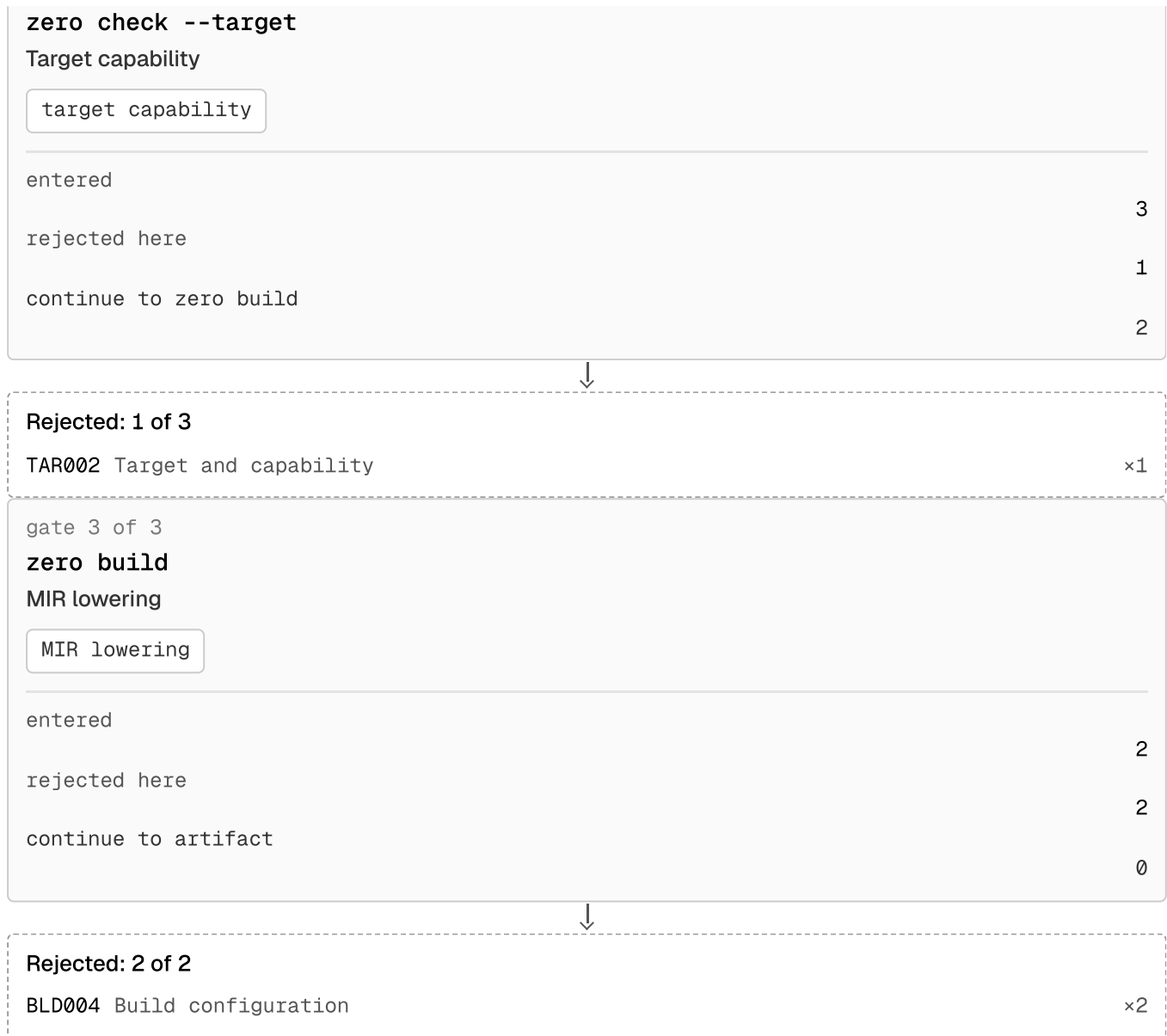
Two structural invariants hold across the corpus. The `sourceMap` row count equals the `node` count exactly for every program — every graph node retains a source position. And the edge count is exactly two fewer than the node count, consistent with a spanning structure over the module and package roots.

6.4 Error handling: three admission gates, not one

This is the paper's central empirical result. We expected the error corpus to partition by phase within a single compile. Instead it partitions by *gate*, and there are three.

10 deliberately malformed programs enter at the left. Each gate admits what it cannot fault and turns the rest away downward.





Counts and codes are derived at render time from `capture.errorCases`: a case is attributed to the gate its `rejectedAt` field names, and the codes are the diagnostics that gate itself emitted. 0 of 10 malformed programs reach an artifact. On a narrow screen the gates stack, so the left-to-right flow becomes top to bottom; the rejected branch stays directly beneath its gate either way.

Text alternative to the diagram above: the three admission gates in order, with the population entering each, the number it rejected, the number it passed on, and the diagnostic codes it rejected them with. Population: the 10 error-corpus programs. Counts are programs, not diagnostic lines.

#	Gate	Checks	Entered	Rejected	Continued	Codes rejected with
1	zero import	lexical, syntax, name resolution, type, mutability, effect, memory	10	7	3	PAR100 ×2, ERR003 ×1, MEM003 ×1, NAM003 ×1, TYP002 ×1, TYP009 ×1
2	zero check --target	target capability	3	1	2	TAR002 ×1
3	zero build	MIR lowering	2	2	0	BLD004 ×2
Reached an artifact				—	0	—

The 10 deliberately malformed programs in the error corpus, one row each. The three gate columns record the outcome of `zero import`, `zero check` and `zero build`: “ok” passed, “rejected” was refused, — was never reached. No units.

Case	Targets phase	Import	Check	Build	Rejected at	Codes
e01_lexical	lexical	rejected	ok	—	import	PAR100
e02_syntax	syntax	rejected	ok	—	import	PAR100
e03_name	resolve	rejected	ok	—	import	NAM003
e04_type	check	rejected	ok	—	import	TYP002
e05_mutability	check	rejected	ok	—	import	TYP009
e06_effect	check	rejected	ok	—	import	ERR003
e07_memory	check	rejected	ok	—	import	MEM003
e08_target	target	ok	rejected	rejected	check	TAR002
e09_lowering	lower	ok	ok	rejected	build	BLD004
e10_match	—	ok	ok	rejected	build	BLD004

The 7 front-end failures were all refused at `zero import`. Critically, `zero check` subsequently reported `ok` for all of them, because the malformed program never entered the store — after each rejection `zero view --fn main` still projected the *previous* program. The capability violation passed ingestion and was refused by `zero check --target`. And 2 cases passed both and were refused only at `zero build`.

Which gate stopped it, and what it said

Pick one of the 10 deliberately malformed programs. Zero offers three places to refuse it — admitting text into the graph, checking the stored graph, and building an artifact — and each panel below reports what that gate actually returned for this case.

Error case

e01_lexical lexical	e02_syntax syntax	e03_name resolve	e04_type check	e05_mutability check	e06_effect check
e07_memory check	e08_target target	e09_lowering lower	e10_match no declared phase		

e01_lexical

A byte the scanner cannot classify into any token kind.

targets phase lexical · expected code PAR100 · target host default · rejected at import · codes PAR100

Source as submitted

src/main.0

5 lines · 114 B

```
pub fn main(world: World) -> Void raises {
  let bad: i32 = 4 § 2
  check world.out.write("unreachable\n")
}
```

► Graph store after the attempt — storeContaminated: false

Three gates, in order

1 zero import

■ rejected

Admits text into the graph. Scanning, parsing, name binding, type, effect and frame-budget checking all run here, before anything is written to zero.graph.

This is the gate that turned the program away.

error · PAR100 · Parsing

unexpected token in expression

Location e01_lexical/src/main.0:2:
22

Expected expression

Actual Â

Help use canonical.0 text source

Fix safety requires-human-review

Repair repair-syntax — Repair the syntax at the reported parser span, then rerun zero check.

2 zero check

○ passed

Re-checks the stored graph and reports target readiness alongside a top-level verdict.

Ran after the earlier rejection, so it judged the graph as it then stood, not the submitted text.

```
ok: true
targetReadiness.ok: true
targetReadiness.buildable: true
targetReadiness.stage: ready
```

No diagnostic recorded at this gate.

3 zero build

□ not reached

Lowers the checked graph to MIR and emits an artifact for the selected target.

No diagnostic recorded at this gate.

The same three verdicts as a table, for the case shown above. Values are the raw importOk, checkOk and buildOk fields: — means the capture recorded null, which is the gate never being reached. No units.

Gate	Command	Field	Value	Outcome	Diagnostics
------	---------	-------	-------	---------	-------------

Gate	Command	Field	Value	Outcome	Diagnostics
1	zero import	importOk	false	rejected — rejected here	1
2	zero check	checkOk	true	passed	0
3	zero build	buildOk	—	not reached	0

Each diagnostic above is the compiler's own record, not a rendering of its prose: a stable code, a span, an expected and an actual fact, a fix-safety rating and, where the compiler has one, a named repair. That is what makes a rejection something a program can act on rather than a paragraph a human must read.

6.5 The front end and the back end accept different languages

We built every corpus program for every advertised target: 64 combinations. 47 succeeded (73%). Every one of the 17 failures was BLD004 — the back end declining to lower a construct semantic analysis had accepted.

Build outcome for every corpus program on every target the installed toolchain advertises (8 programs × 8 targets = 64 builds, 17 of them refused). A cell holding a number is the emitted artifact size in bytes; a cell holding a diagnostic code is a refusal, and the code is the marker.

Program	darwin-arm64	darwin-x64	linux-musl-x64	linux-musl-arm64	linux-x64	linux-arm64	win32-x64.exe	win32-arm64.exe
p01_hello	16,632	16,636	352	312	352	312	1,536	1,536
p02_arith	16,632	16,636	633	649	633	649	2,048	2,048
p03_control	16,632	16,636	795	867	795	867	2,048	2,048
p04_shapes	16,632	16,636	826	BLD004	826	BLD004	2,048	BLD004
p05_errors	16,632	BLD004	826	BLD004	826	BLD004	BLD004	BLD004
p06_memory	16,632	16,636	2,439	BLD004	2,439	BLD004	3,584	BLD004
p07_generics	16,632	16,636	588	BLD004	588	BLD004	1,536	BLD004
p08_lexer	16,632	16,636	4,237	BLD004	4,237	BLD004	5,632	BLD004
Built	8/8	7/8	8/8	3/8	8/8	3/8	7/8	3/8

BLD004 · record — 9 builds

BLD004 · IR_VALUE_CHECK — 4 builds

BLD004 · unsupported instruction — 3 builds

BLD004 · IR_VALUE_RESCUE — 1 build

The failures name specific constructs the MIR subset cannot represent: aggregate values crossing a function boundary, `check` and `rescue` on user-defined fallible functions, and instructions absent from an architecture's emitter.

First, **backend completeness is target-specific**. The `p05_errors` program fails to build on the Windows host but cross-compile to an 826-byte ELF for `linux-musl-x64`. Same graph, same front end, two answers.

Second, a passing `zero check` does not imply a buildable program — though the compiler is not ignorant. It records `targetReadiness.buildable: false`, `stage: "lower"` and a BLD004 naming the construct. The difficulty is that the same document carries `ok: true` with an empty top-level `diagnostics` array. A consumer testing the field the schema presents as the verdict gets the wrong answer.

Table 6. Fields reported by `zero check --json` for the two lowering cases. Both rows describe the same compilation.

Case	Construct	ok	diagnostics	buildable	stage	Actual build
e09_lowering	Outcome	true	0	false	lower	BLD004
e10_match	Match	true	0	false	lower	BLD004

The 8 targets the installed toolchain advertises, against the union of 9 capability names declared across them. “yes” means the target declares that capability; — means it does not. No units.

Target	OS	Arch	Object format	libc	args	env	fs	memory	net	proc	rand	stdio	time	Declared
darwin-arm64	macos	aarch64	macho	default	yes	yes	yes	yes	yes	yes	yes	yes	yes	9
darwin-x64	macos	x86_64	macho	default	—	—	yes	yes	—	—	yes	yes	yes	5
linux-musl-x64	linux	x86_64	elf	musl	yes	yes	yes	yes	—	—	yes	yes	yes	7
linux-musl-arm64	linux	aarch64	elf	musl	—	—	—	yes	—	—	yes	yes	yes	4
linux-x64	linux	x86_64	elf	gnu	—	—	—	yes	—	—	yes	yes	yes	4
linux-arm64	linux	aarch64	elf	gnu	—	—	—	yes	—	—	yes	yes	yes	4
win32-x64.exe	windows	x86_64	coff	msvc	yes	yes	yes	yes	yes	yes	yes	yes	yes	9
win32-arm64.exe	windows	aarch64	coff	msvc	—	—	—	yes	—	—	yes	yes	yes	4

7. The corpus, in full

All 8 programs are reproduced below with their measurements, their test results and their real output. Together they are 680 non-empty lines of Zero. Each was written to exercise a distinct set of compiler tables.

7.1 Watching one compile

A replay of the recorded measurements, stepped through phase by phase. This is not a live compile — it is the captured data for the selected program, advanced one reported phase at a time.

8. Interactive phase explorer

Take one program and watch it become each successive representation. Every panel shows real captured output from the command named in its caption.

One program through six phases

Pick a program and a phase. Each panel shows the artifact Zero 0.3.4 emitted for that program at that phase, together with the command that produced it.

Program

p01_hello - 8 lines ▼

Line counts cover every file in the package; the source shown below is src/main.0.



```
src/main.0

pub fn main(world: World) -> Void raises {
  check world.out.write("hello from zero\n")
}
```

8 Source lines, all files	41 Tokens
13 Graph nodes	1,536B Artifact bytes

1 Lexical analysis

Character stream → Token stream. Carried by parse.

Token stream

src/main.0 · 26 tokens captured

```
pub fn main ( world : World ) -> Void raises {
  check world . out . write ( "hello from zero\n" )
}
```

Token counts by kind. 41 tokens across 2 files, of which 35 are code tokens.

Kind	src/lib.0	src/main.0	Total
word	5	11	16
symbol	5	10	15
number	1	0	1
string	0	1	1
newline	3	3	6
eof	1	1	2
All kinds	15	26	41

Output of `zero tokens --json`: the character stream of `src/main.0` classified into 6 kinds, counted per file.

The interactive explorer is available in the web edition at zero.khe.money.

9. Playground

Write Zero and see it analysed. Lexing and parsing run in your browser: we reimplemented Zero's scanner and declaration parser in TypeScript, then validated them against the compiler's own `zero tokens --json` output. Agreement is exact — kind, text, line and column — on all 8 corpus programs, 2,666 tokens in total, and the parser recovers the same function set as `zero parse --json` on all eight. Those two phases are therefore real, not simulated, and they run on whatever you type.

Phases three to six cannot run here. Zero 0.3.4 ships no WebAssembly target and its own `selfHostRouting` report marks `browserCompiler` as removed, so for the corpus programs those phases are replayed from the recorded capture — and the terminal says so explicitly once you edit the buffer away from a captured program.

The playground is available in the web edition at zero.khe.money.

11. Discussion

11.1 Phases relocate; they do not disappear

It would be easy to read Zero's architecture as abolishing the front end. It does not. Every classical phase is present and every one still runs — but the first three moved out of the compile path into an ingestion gate that runs once per edit rather than once per build.

This relocation explains all our findings at once: the timing distribution in §6.2, the containment result in §6.4, and the acceptance gap in §6.5. One architectural choice, three consequences.

11.2 Two levels of truth in one document

Our first reading of the acceptance gap was wrong: we recorded it as the compiler failing to detect a problem. It detects it. `targetReadiness` carries the correct verdict and a diagnostic naming the offending construct.

What the compiler does is subtler and, for its stated audience, arguably worse than not checking: it publishes two summaries of the same compilation that disagree. A human reading the whole document notices. A program reading the field the schema presents as the verdict does not. A language whose central claim is that output should be consumed by machines has taken on an obligation a human-facing compiler has not: its top-level fields are an API. Machine-readability is necessary but not sufficient for machine-*reliability*.

11.3 Implications for compiler pedagogy

Our practical conclusion is narrower than Zero's marketing and, we think, more durable. A student using Zero can print the phase list, time each phase, read the symbol table as sixteen relations, watch a program be refused at three distinct gates, and diff the object formats produced by five emitters from one source graph — from the command line, without patching a compiler. We are not aware of another production-intent compiler where the phase structure is this directly inspectable.

13. Threats to validity

Single version, single platform. All measurements are from Zero 0.3.4 build 5b3a90a on one Windows x64 machine. Zero is explicitly experimental and shipped four minor versions in roughly a month; the backend gaps we document are the ones most likely to close.

Corpus scale. 680 lines across 8 programs is small. It is adequate for demonstrating phase structure and the acceptance gap, both qualitative properties, but too small to resolve front-end phase timings against the compiler's 1 ms granularity or to claim asymptotic scaling.

Corpus authorship. The corpus was written to exercise particular compiler tables, so coverage is deliberate rather than representative. Several programs were shaped by what the backend would accept, which biases them toward the lowerable subset — if anything this *understates* the acceptance gap.

Timing methodology. Wall-clock includes ~40 ms of process startup we could not separate without instrumenting the binary, so all phase-level claims rest on the compiler's self-reported times.

Browser reimplementations. The playground's lexer and parser are ours, not Zero's. They agree exactly with `zero tokens --json` across all 8 corpus programs, but that corpus is 680 lines written by us: agreement on it is evidence of correctness on the constructs we used, not a proof of equivalence. Input using syntax the corpus never exercises may diverge, and only the native compiler is authoritative.

Documentation discrepancies. Several examples in the shipped language guide do not compile on this build. We report these as observations about version 0.3.4, not claims about the language design.

14. Related work

The phase decomposition we test against is due to Aho, Lam, Sethi and Ullman (<https://www.pearson.com/en-us/subject-catalog/p/compilers-principles-techniques-and-tools/P200000003472>), with incremental and query-driven alternatives surveyed by Cooper and Torczon (<https://www.elsevier.com/books/engineering-a-compiler/cooper/978-0-12-815412-0>). Zero's persistent-graph approach has clear antecedents: the Roslyn (<https://github.com/dotnet/roslyn>) compiler platform exposed compilation as a queryable object model, and query-based architectures — rustc's demand-driven query system (<https://rustc-dev-guide.rust-lang.org/query.html>) and the salsa (<https://github.com/salsa-rs/salsa>) framework behind Rust Analyzer — already treat compilation as incremental recomputation over a memoised fact database. Zero's distinguishing move is making the graph the *authoritative stored artifact*, with text demoted to a projection.

On structured diagnostics, both rustc (<https://doc.rust-lang.org/rustc/json.html>) and TypeScript ship machine-readable error output; Zero's contribution is typed repair identifiers and safety ratings rather than machine-readability itself.

We are not aware of prior empirical work measuring a compiler's phase structure through its own reporting interface, largely because few compilers expose one.

16. Conclusion

The classical six-phase model survives contact with a graph-first compiler, but not in the shape the diagram suggests. Across 8 programs and 64 build combinations we find that Zero implements every classical phase while relocating the first three out of the compile path into an ingestion gate. That single change explains our measurements: lowering accounts for 100% of reported phase time; all 7 front-end error cases were contained at the gate; and 17 of 64 builds failed on programs the front end had accepted.

For a compilers course, the practical finding is that Zero makes the phases *visible* in a way mainstream toolchains do not. Whether the language succeeds on its own agent-oriented terms is a separate question, whose answer probably has more to do with training-data distribution than with compiler design. The observability is worth studying either way.

References

A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman. *Compilers: Principles, Techniques, and Tools*. 2nd ed., Pearson, 2007. [pearson.com](https://www.pearson.com)

K. D. Cooper, L. Torczon. *Engineering a Compiler*. 3rd ed., Morgan Kaufmann, 2022. [elsevier.com](https://www.elsevier.com)

Vercel Labs. *Zero — the programming language for agents*. zerolang.ai · github.com/vercel-labs/zerolang (Apache-2.0). Version 0.3.4, build 5b3a90a.

Vercel Labs. *Graph architecture*. zerolang.ai/concepts/graph-architecture

Vercel Labs. *Compile path*. zerolang.ai/concepts/compile-path

Vercel Labs. *Semantic graph vs text*. zerolang.ai/concepts/semantic-vs-text

Vercel Labs. *Getting started*. zerolang.ai/getting-started. Version-matched language documentation retrieved via `zero skills get language --full`.

Microsoft. *.NET Compiler Platform (Roslyn)*. github.com/dotnet/roslyn

N. Matsakis et al. *The rustc query system*. rustc-dev-guide.rust-lang.org/query.html

The Rust Project. *JSON output*. doc.rust-lang.org/rustc/json.html

salsa-rs. *salsa — on-demand, incrementalized computation*. github.com/salsa-rs/salsa

Vercel Labs. *wterm — a DOM-based terminal emulator with a WebAssembly core*. github.com/vercel-labs/wterm. Reviewed as a delivery option; see A.1.

Vercel. *Geist and Web Interface Guidelines*. vercel.com/geist · vercel.com/design/guidelines, with WCAG 2.2 AA for contrast and structure.

Appendix A. Reproduction

Everything is regenerated from one command. Source and dataset: github.com/HKTITAN/phases-of-zero (<https://github.com/HKTITAN/phases-of-zero>).

```
regenerate everything
```

```
curl -fsSL https://zerolang.ai/install.sh | bash
export PATH="$HOME/.zero/bin:$PATH"
zero --version          # 0.3.4 (build 5b3a90a)

npm install
npm run paper           # capture -> qr -> build -> pdf -> epub -> previews
```

A.1 Why the explorer ships precomputed data

We considered compiling Zero in the browser, following wterm (<https://github.com/vercel-labs/wterm>), which runs a Zig-authored terminal core as WebAssembly. Zero 0.1.3 advertised `wasm32-wasi` and `wasm32-web` targets. Version 0.3.4 advertises neither: the target list contains 8 native targets and no WebAssembly target, and the compiler's own `selfHostRouting` report marks `browserCompiler` as removed. We therefore reimplemented phases one and two in TypeScript for the playground and replay the rest from the capture.

A.2 Corpus contents

p01_hello, p02_arith, p03_control, p04_shapes, p05_errors, p06_memory, p07_generics, p08_lexer under `corpus/`, and e01_lexical, e02_syntax, e03_name, e04_type, e05_mutability, e06_effect, e07_memory, e08_target, e09_lowering, e10_match under `errors/`.

Harshit Khemani, Kush Ahuja, Mohit Kumar Mishra, Kushagra Agrawal · zero.khe.money

Zero 0.3.4 · 2026-08-09